

透過 Slurm 部署可自動調整資源配置的 HPC 叢集

來源：<https://codelabs.developers.google.com/codelabs/hpc-slurm-on-gcp?hl=zh-tw>

步驟數：8

瞭解如何使用 Google Compute Engine、Google Deployment Manager 和 Slurm Workload Manager 佈建可動態擴充的 HPC 叢集。

目錄

1. [總覽](#)
2. [設定](#)
3. [準備及檢查 Slurm Terraform 設定](#)
4. [部署及驗證設定](#)
5. [登入 Slurm 叢集](#)
6. [執行 Slurm 工作並擴充叢集](#)
7. [執行 MPI 工作](#)
8. [結論](#)

步驟 1 · 約 0 分鐘

1. 總覽

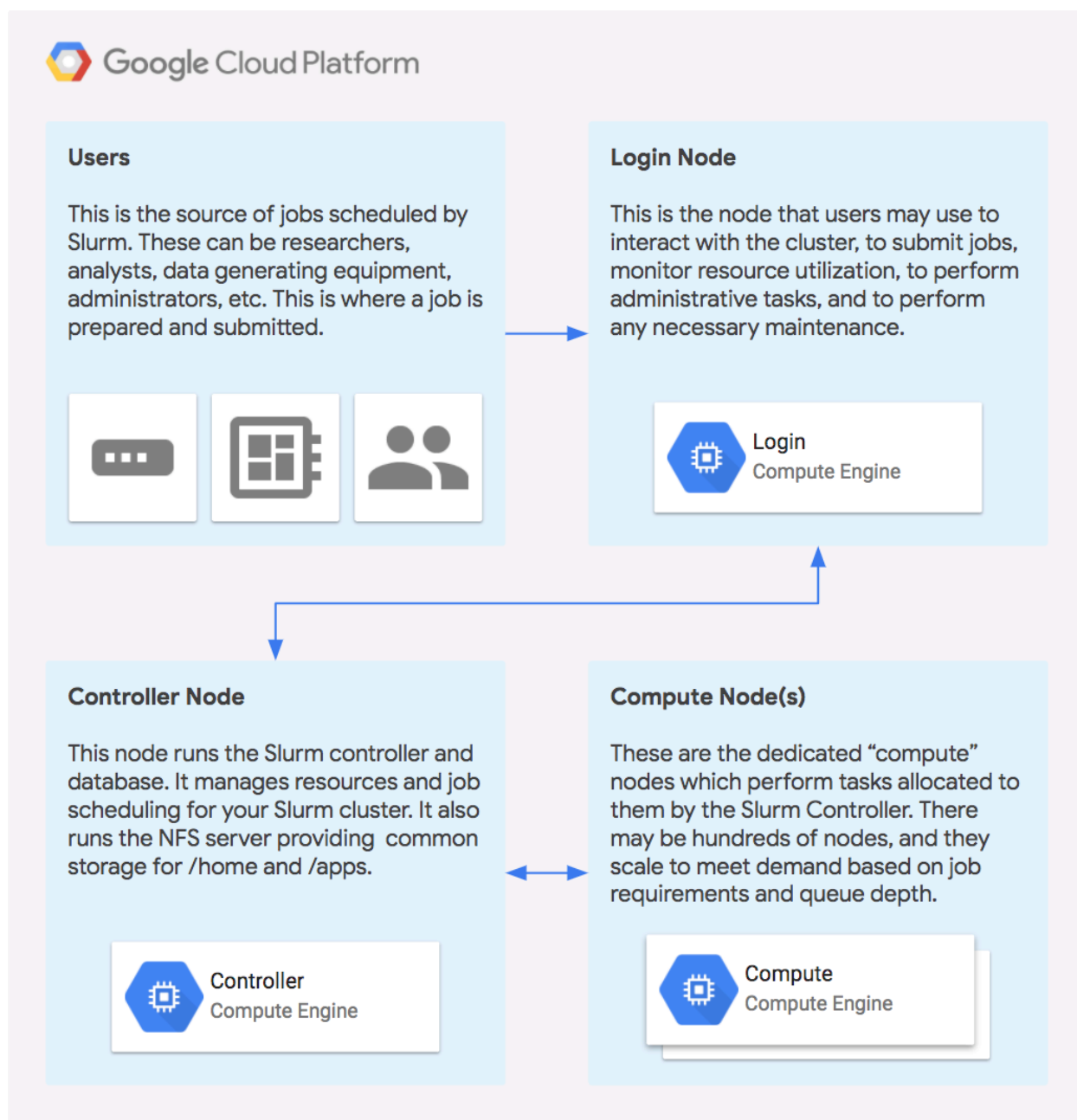
歡迎參加 Google 程式碼研究室，瞭解如何在 Google Cloud Platform 上執行 Slurm 叢集！完成本程式碼研究室後，您應該會充分瞭解如何輕鬆佈建及操作可自動調整資源配置的 Slurm 叢集。



Google Cloud 與 [SchedMD](#) 合作推出一系列工具，讓您更輕鬆地在 Compute Engine 中啟動 Slurm 工作負載管理員，並在需要額外資源時動態擴充現有叢集。這項整合功能是由 [SchedMD](#) 專家按照 Slurm 最佳做法建構而成。

如果您打算使用 [Google Cloud Platform 上的 Slurm](#) 整合功能，或有任何疑問，請考慮加入 [Google Cloud 和 Slurm 社群討論群組](#)！

關於 Slurm



Google Cloud Platform 中獨立 Slurm 叢集的基本架構圖。

Slurm 是全球 HPC 叢集的主要工作負載管理工具之一。Slurm 提供開放原始碼、容錯和高擴充性的工作負載管理與工作排程系統，適用於小型和大型 Linux 叢集。Slurm 運作時不需要修改核心，且相對獨立。Slurm 是叢集工作負載管理員，具有三項主要功能：

1. 這項服務會將資源 (運算節點) 的專屬或非專屬存取權分配給使用者一段時間，方便他們執行工作。
2. 這個框架可讓您在分配到的節點上啟動、執行及監控工作 (通常是平行工作)。
3. 並管理待處理工作的佇列，仲裁資源爭用情形。

進一步瞭解 Slurm

- [SchedMD 首頁](#)
- [GCP 上的 Slurm 讀我檔案](#)
- [Slurm 快速入門指南](#)
- [Slurm MAN 頁面](#)
- [Slurm 指令摘要 \(PDF\)](#)
- [Slurm 會計指南](#)
- [Slurm 疑難排解指南](#)
- [在 GCP 上的 Slurm 安裝應用程式](#)
- [Slurm 使用者討論群組](#)

課程內容

- 如何使用 Terraform 設定 Slurm 叢集
- 如何使用 SLURM 執行作業
- 如何查詢叢集資訊，以及監控 SLURM 中執行的工作
- 如何自動調度節點，以配合特定工作參數和需求
- 如何取得 Slurm 相關說明

必要條件

- Google Cloud Platform 帳戶和專案 (已啟用帳單功能)
 - 具備基本 Linux 經驗
-

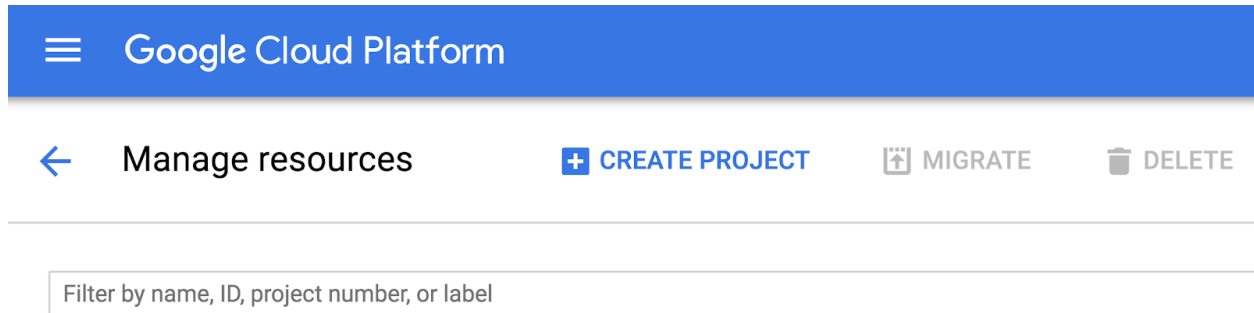
步驟 2 · 約 2 分鐘

2. 設定

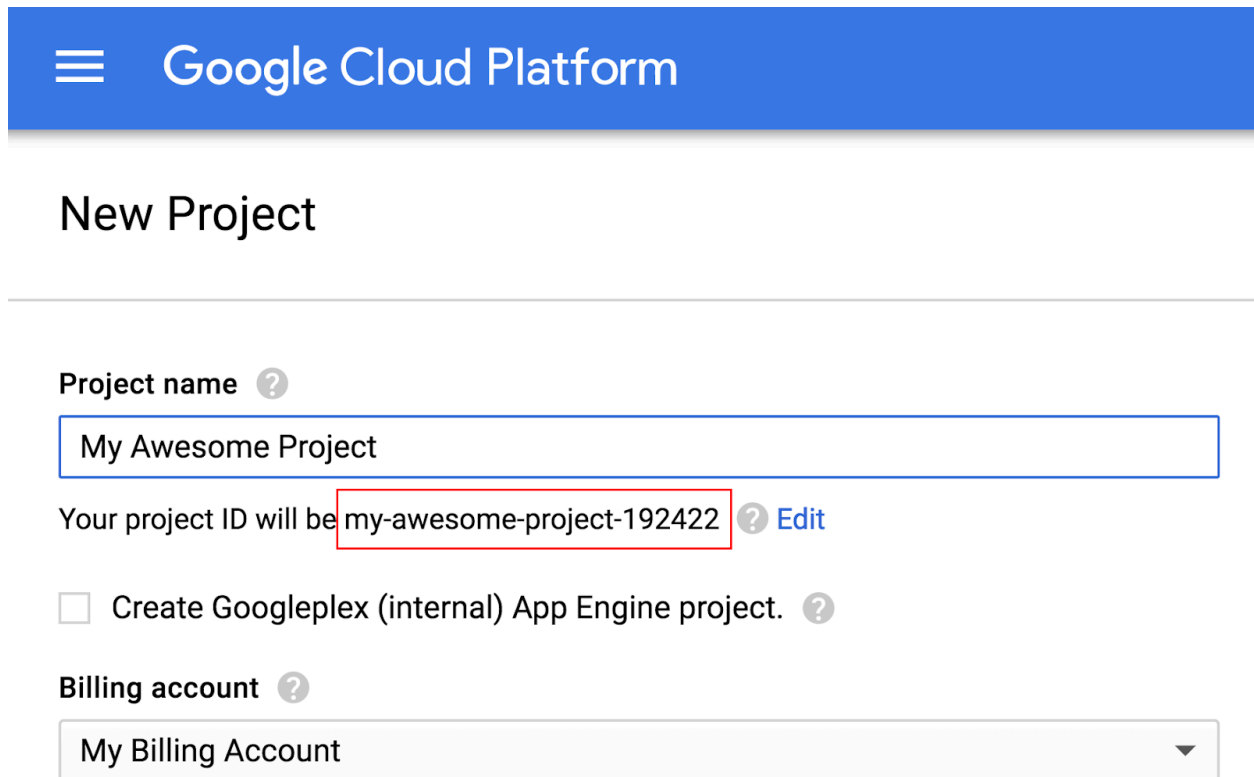
自修實驗室環境設定

建立專案

如果沒有 Google 帳戶 (Gmail 或 G Suite)，請先[建立帳戶](#)。登入 Google Cloud Platform 主控台 (console.cloud.google.com)，然後開啟「[管理資源](#)」頁面：



按一下「建立專案」。



輸入專案名稱。記下專案 ID (上圖螢幕截圖中以紅色醒目顯示)。所有 Google Cloud 專案的專案 ID 都是不重複的名稱。如果專案名稱不是唯一的，Google Cloud 會根據專案名稱隨機產生專案 ID。

接著，您需要在 Developers Console 中[啟用帳單](#)，才能使用 Google Cloud 資源。

完成這項程式碼研究室的費用不應超過數美元，但如果您決定使用更多資源，或是將資源繼續執行 (請參閱本文件結尾的「結論」一節)，則可能會增加費用。如要使用 Google Cloud Platform Pricing Calculator，請前往[這個頁面](#)。

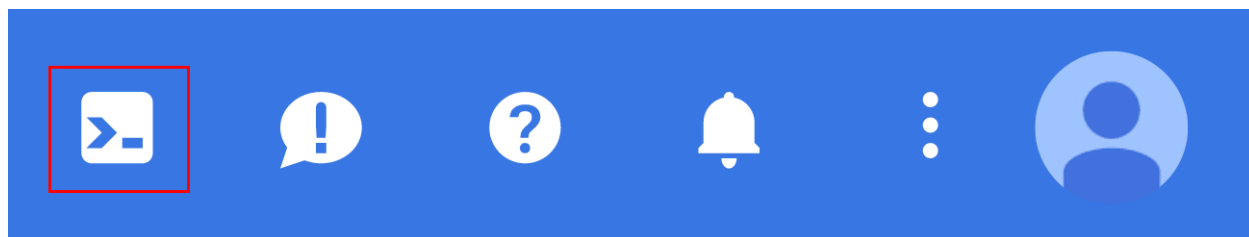
Google Cloud Platform 新使用者享有價值 **\$300 美元**的免費試用期。

Google Cloud Shell

雖然可以透過筆電遠端操作 Google Cloud，但在本程式碼實驗室中，我們將使用 [Google Cloud Shell](#)，這是可在雲端執行的指令列環境。

啟動 Google Cloud Shell

在 GCP 主控台，按一下右上角工具列的 Cloud Shell 圖示：



然後按一下「啟動 Cloud Shell」：

Google Cloud Shell

Free, pre-installed with the tools you need for the Google Cloud Platform. [Learn More](#)

```
example-vm-2    europe-west1-b f1-micro          10.240.119.112 104.155.36.122 RUNN
example-vm-3    us-central1-f  f1-micro          10.240.57.1   104.154.76.241 RUNN
google77703_student@cloudshell:~$
google77703_student@cloudshell:~$ git clone https://github.com/GoogleCloud/appengine-examp
Cloning into 'appengine-example'...
remote: Counting objects: 476, done.
remote: Total 476 (delta 0), reused 0 (delta 0), pack-reused 476
Receiving objects: 100% (476/476), 432.65 KiB | 0 bytes/s, done.
Checking connectivity... done.
google77703_student@cloudshell:~$ cd appengine-example
google77703_student@cloudshell:~/appengine-example$
```

Real Linux environment	Configured for Google Cloud	Popular language support
- Linux Debian-based OS 5GB persisted home directory - Add, edit and save files	- Google Cloud SDK - Google App Engine SDK - Docker - Git - Text editors - Build tools View more	- Python - Java - Go - Node.js

[CANCEL](#) [START CLOUD SHELL](#)

佈建並連線至環境的作業需要一些時間才能完成：



這部虛擬機器搭載各種您需要的開發工具，並提供永久的 5GB 主目錄，而且可在 Google Cloud 運作，大幅提升網路效能並簡化驗證程序。本實驗室幾乎所有工作都可在網頁瀏覽器或 Google Chromebook 上完成。

連至 Cloud Shell 後，您應該會看到驗證已完成，專案也已設為獲派的「PROJECT_ID」`PROJECT_ID`：

```
$ gcloud auth list
```

指令輸出：

```
Credentialed accounts:  
- <myaccount>@<mydomain>.com (active)
```

```
$ gcloud config list project
```

指令輸出：

```
[core]  
project = <PROJECT_ID>
```

如果專案 ID 未正確設定，請輸入下列指令手動設定專案：

```
$ gcloud config set project <PROJECT_ID>
```

指令輸出：

```
Updated property [core/project].
```

注意：**gcloud** 是 Google Cloud Platform 功能強大的整合式指令列工具，如需完整說明文件，請前往 <https://cloud.google.com/sdk/gcloud>。並已預先安裝於 Cloud Shell。你會發現它支援 Tab 鍵自動完成功能。

步驟 3 · 約 5 分鐘

3. 準備及檢查 Slurm Terraform 設定

下載 Slurm Terraform 設定

在 Cloud Shell 工作階段中，執行下列指令來複製 (下載) Git 存放區，其中包含適用於 Google Cloud Platform 的 Slurm Terraform 檔案：

```
git clone https://github.com/SchedMD/slurm-gcp.git
```

執行下列指令，切換至 Slurm 部署作業設定目錄：

```
cd slurm-gcp
```

設定 Slurm Terraform tfvars

`basic.tfvars.example` 檔案詳細說明部署作業的設定，包括要部署的網路、執行個體和儲存空間。將其複製到新檔案 (我們將其稱為「tfvars 檔案」)，然後視需要編輯。

```
cd tf/example/basic  
cp basic.tfvars.example basic.tfvars
```

在 Cloud Shell 工作階段中，開啟 tfvars 檔案 `basic.tfvars`。您可以使用偏好的指令列編輯器 (`vi`、`nano`、`emacs` 等)，或使用 Cloud Console 程式碼編輯器查看檔案內容：



查看 tfvars 檔案的內容。

```

cluster_name = "g1"
project      = "<project>"
zone         = "us-west1-b"

# network_name          = "<existing network name>"
# subnetwork_name       = "<existing subnetwork name>"
# shared_vpc_host_project = "<vpc host project>"

# disable_controller_public_ips = true
# disable_login_public_ips      = true
# disable_compute_public_ips    = true

# suspend_time = 300

controller_machine_type = "n1-standard-2"
controller_image         = "projects/schedmd-slurm-public/global/images/family/schedmd-slurm-20-11-4-hpc-centos-7"
controller_disk_type     = "pd-standard"
controller_disk_size_gb = 50
# controller_labels = {
#   key1 = "val1"
#   key2 = "val2"
# }
# controller_service_account = "default"
# controller_scopes          = [ "https://www.googleapis.com/auth/cloud-platform" ]
# cloudsql = {
#   server_ip = "<cloudsql ip>"
#   user      = "slurm"
#   password  = "verysecure"
#   db_name   = "slurm_accounting"
# }
# controller_secondary_disk      = false
# controller_secondary_disk_size = 100
# controller_secondary_disk_type = "pd-ssd"
#
# When specifying an instance template, specified controller fields will
# override the template properties.
# controller_instance_template = null

login_machine_type = "n1-standard-2"
login_image         = "projects/schedmd-slurm-public/global/images/family/schedmd-slurm-20-11-4-hpc-centos-7"
login_disk_type     = "pd-standard"
login_disk_size_gb = 20
# login_labels = {
#   key1 = "val1"
#   key2 = "val2"
# }
# login_node_count = 1
# login_node_service_account = "default"
# login_node_scopes          = [
#   "https://www.googleapis.com/auth/monitoring.write",

```

```

# "https://www.googleapis.com/auth/logging.write"
# ]
#
# When specifying an instance template, specified login fields will
# override the template properties.
# login_instance_template = null

# Optional network storage fields
# network_storage is mounted on all instances
# login_network_storage is mounted on controller and login instances
# network_storage = [{
#   server_ip      = "<storage host>"
#   remote_mount   = "/home"
#   local_mount    = "/home"
#   fs_type        = "nfs"
#   mount_options  = null
# }]
#
# login_network_storage = [{
#   server_ip      = "<storage host>"
#   remote_mount   = "/net_storage"
#   local_mount    = "/shared"
#   fs_type        = "nfs"
#   mount_options  = null
# }]

# compute_node_service_account = "default"
# compute_node_scopes          = [
#   "https://www.googleapis.com/auth/monitoring.write",
#   "https://www.googleapis.com/auth/logging.write"
# ]

partitions = [
  { name              = "debug"
    machine_type      = "n1-standard-2"
    static_node_count = 0
    max_node_count    = 10
    zone              = "us-west1-b"
    image              = "projects/schedmd-slurm-public/global/images/family/schedmd-slurm-
20-11-4-hpc-centos-7"
    image_hiperthreads = false
    compute_disk_type  = "pd-standard"
    compute_disk_size_gb = 20
    compute_labels      = {}
    cpu_platform       = null
    gpu_count          = 0
    gpu_type           = null
    network_storage     = []
    preemptible_bursting = false
    vpc_subnet         = null
    exclusive          = false
    enable_placement    = false
  }
]

```

```

    regional_capacity    = false
    regional_policy      = {}
    instance_template    = null
  },
  # { name                = "partition2"
  #   machine_type        = "n1-standard-16"
  #   static_node_count   = 0
  #   max_node_count      = 20
  #   zone                = "us-west1-b"
  #   image                = "projects/schedmd-slurm-public/global/images/family/schedmd-
slurm-20-11-4-hpc-centos-7"
  #   image_hyphertreads  = false
  #
  #   compute_disk_type   = "pd-ssd"
  #   compute_disk_size_gb = 20
  #   compute_labels      = {
  #     key1 = "val1"
  #     key2 = "val2"
  #   }
  #   cpu_platform        = "Intel Skylake"
  #   gpu_count           = 8
  #   gpu_type            = "nvidia-tesla-v100"
  #   network_storage     = [{
  #     server_ip          = "none"
  #     remote_mount       = "<gcs bucket name>"
  #     local_mount        = "/data"
  #     fs_type            = "gcsfuse"
  #     mount_options      = "file_mode=664,dir_mode=775,allow_other"
  #   }]
  #   preemptible_bursting = true
  #   vpc_subnet           = null
  #   exclusive            = false
  #   enable_placement     = false
  #
  #   ### NOTE ###
  #   # regional_capacity is under development. You may see slowness in
  #   # deleting lots of instances.
  #   #
  #   # With regional_capacity : True, the region can be specified in the zone.
  #   # Otherwise the region will be inferred from the zone.
  #   zone = "us-west1"
  #   regional_capacity    = True
  #   # Optional
  #   regional_policy      = {
  #     locations = {
  #       "zones/us-west1-a" = {
  #         preference = "DENY"
  #       }
  #     }
  #   }
  #
  #   When specifying an instance template, specified compute fields will

```

```
#    override the template properties.
#    instance_template = "my-template"
]
```

這個 tfvars 檔案中有數個欄位可供設定。唯一必須設定的欄位是 **project**。範例中的所有其他設定都可以直接使用，但請視情況修改。如要進一步瞭解設定選項，請參閱[這篇文章](#)。

- **cluster_name**: Slurm 叢集名稱
- **project**: 要部署資源的 Google Cloud 專案 ID
- **可用區**: 這個叢集的控制器和登入執行個體所在的 Google Cloud 可用區 - [瞭解詳情](#)
- **network_name**: 要將 Slurm 叢集部署至的[虛擬私有雲網路](#)
- **subnetwork_name**: [虛擬私有雲子網路](#)，用於部署 Slurm 叢集
- **shared_vpc_host_project**: 要將 Slurm 叢集部署至的[共用虛擬私有雲網路](#)
- **disable_controller_public_ips**: 是否要將外部 IP 指派給 Slurm 控制器？
- **disable_login_public_ips**: 是否要將外部 IP 指派給 Slurm 登入節點？
- **disable_compute_login_ips**: 是否要將外部 IP 指派給 Slurm 登入節點？
- **suspend_time**: 節點閒置多久後會暫停
- **controller_machine_type**: 控制器節點[執行個體類型](#)
- **controller_image**: 用於建立 Slurm 控制器執行個體的 GCP 映像檔
- **controller_disk_type**: 控制器執行個體開機磁碟類型
- **controller_disk_size_gb**: 控制器執行個體開機磁碟的大小
- **controller_labels**: 要附加至控制器執行個體的標籤
- **controller_service_account**: 要在控制器執行個體上使用的[服務帳戶](#)
- **controller_scopes**: 控制器的執行個體[存取權範圍](#)
- **cloudsql**: [Google CloudSQL](#) 伺服器，用來做為 Slurm 資料庫，而非在控制器執行個體上代管資料庫
- **server_ip**: CloudSQL 伺服器 IP
- **user**: Cloud SQL 使用者名稱
- **password**: Cloud SQL 密碼
- **db_name**: CloudSQL 資料庫名稱
- **controller_secondary_disk**: 要新增次要磁碟做為 NFS 伺服器儲存空間嗎？
- **controller_secondary_disk_type**: 控制器次要磁碟類型
- **controller_secondary_disk_size_gb**: 控制器次要磁碟的大小
- **controller_instance_template**: 用於控制器執行個體的 [GCP 執行個體範本](#)。指定的任何運算欄位都會覆寫範本屬性。舉例來說，如果指定 controller_image，系統會覆寫執行個體範本中的映像檔。
- **login_machine_type**: 登入 (可透過 SSH 存取) 節點的[執行個體類型](#)
- **login_image**: 用於建立 Slurm 登入執行個體的 GCP 映像檔
- **login_disk_type**: 登入執行個體開機磁碟類型
- **login_disk_size_gb**: 登入執行個體開機磁碟的大小
- **login_labels**: 要附加至登入執行個體的標籤
- **login_node_count**: 要建立的登入節點數
- **login_node_service_account**: 要在登入執行個體上使用的[服務帳戶](#)
- **login_node_scopes**: 登入執行個體的[存取權範圍](#)

- **login_instance_template**:用於登入執行個體的 [GCP 執行個體範本](#)。指定的任何運算欄位都會覆寫範本屬性。舉例來說，如果指定了 login_image，系統會覆寫執行個體範本中的圖片。
- **network_storage**:要掛接至所有節點的網路儲存空間。系統會直接將欄位新增至 fstab。如要新增其他掛接點，請重複上述步驟。
- **server_ip**:儲存伺服器 IP
- **remote_mount**:儲存空間掛接名稱 (檔案系統名稱)
- **local_mount**:本機掛接目錄
- **fs_type**:檔案系統類型 (NFS、CIFS、Lustre、GCSFuse 會自動安裝)
- **mount_options**:掛接選項 (即 defaults,_netdev)
- **login_network_storage**:要掛接至登入和控制器節點的網路儲存空間。系統會自動安裝 NFS、CIFS、Lustre 和 GCSFuse。如要新增其他掛接點，請重複上述步驟。
- **server_ip**:儲存伺服器 IP
- **remote_mount**:儲存空間掛接名稱 (檔案系統名稱)
- **local_mount**:本機掛接目錄
- **fs_type**:檔案系統類型 (NFS、CIFS、Lustre、GCSFuse 會自動安裝)
- **mount_options**:掛接選項 (即 defaults,_netdev)
- **compute_node_service_account**: 要在運算執行個體上使用的[服務帳戶](#)
- **compute_node_scopes**: 運算執行個體的[存取權範圍](#)
- **partitions** : Slurm 分區設定。可重複執行以新增其他分割區。
- **name**: 分區名稱
- **machine_type**:運算節點[執行個體類型](#)
- **static_node_count**:隨時可用的運算節點數量
- **max_node_count**:允許的運算節點總數上限，最多 64,000 個
- **可用區**：這個分割區的資源所在 Google Cloud 可用區 - [瞭解詳情](#)
- **image**：運算映像檔節點機型
- **image_hypervisors**:在執行個體上啟用或停用超執行緒
- **compute_disk_type**：運算執行個體開機磁碟類型 (pd-standard、pd-ssd)
- **compute_disk_size_gb**：運算執行個體開機磁碟的大小
- **compute_labels**:要附加至運算執行個體的標籤
- **cpu_platform**:所有運算節點所需的最低 CPU 平台
- **gpu_count**:要附加至分割區中每個執行個體的 GPU 數量
- **gpu_type**: 要附加至分割區執行個體的 [GPU 類型](#)
- **network_storage**:要掛接至分區中所有運算節點的網路儲存空間。系統會直接將欄位新增至 fstab。如要新增其他掛接點，請重複上述步驟。
- **server_ip**:儲存伺服器 IP
- **remote_mount**:儲存空間掛接名稱 (檔案系統名稱)
- **local_mount**:本機掛接目錄
- **fs_type**:檔案系統類型 (NFS、CIFS、Lustre、GCSFuse 會自動安裝)
- **mount_options**:掛接選項
- **preemptible_bursting**:執行個體是否為先占執行個體？
- **vpc_subnet**: 要將 Slurm 分區部署至的[虛擬私有雲子網路](#)

- **exclusive**：啟用 Slurm，將整個節點分配給工作
- **enable_placement**：啟用[配置政策](#)，讓執行個體彼此靠近，縮短執行個體之間的網路延遲時間。
- **regional_capacity**：允許根據可用性，將執行個體放置在該區域的任何可用區
- **regional_policy**：如果 regional_capacity 為 true，這項政策會決定要使用的區域，以及該區域中不得使用的任何可用區
- **Instance_template**：用於運算執行個體的[GCP 執行個體範本](#)。指定的任何運算欄位都會覆寫範本屬性。舉例來說，如果指定了映像檔，就會覆寫執行個體範本中的映像檔。

進階設定

您也可以在叢集部署程序中，選擇安裝其他套件和軟體。您可以使用「[在 Compute Engine 上的 Slurm 叢集中安裝應用程式](#)」一文所述的多種方式，在 Slurm 叢集上安裝軟體，也可以自訂 Slurm 部署的映像檔。目前 Slurm 會部署 SchedMD 提供的 VM 映像檔，該映像檔是以[Google Cloud HPC VM 映像檔](#)為基礎，並在其中安裝 Slurm。

如要使用自己的映像檔，請根據 tfvars 檔案中列出的公開 SchedMD VM 映像檔，建構具有自己設定的映像檔。接著，將 tfvars 檔案中指定的圖片 URI 換成自己的圖片，然後測試變更。

疑難排解

在本程式碼研究室中，請參閱 Slurm-GCP 存放區 ReadMe 的「[疑難排解](#)」一節。

最常見的問題是設定 tfvars 檔案時發生錯誤，以及配額限制。本程式碼研究室的設計，是為了在一般新使用者的標準配額範圍內執行，並使用新使用者獲得的 \$300 美元免費抵免額。如果嘗試建立 VM 失敗，請檢查控制器節點上的 /var/log/slurm/resume.log 檔案，確認是否有 API 錯誤。

步驟 4 · 約 5 分鐘

4. 部署及驗證設定

部署設定

在 Cloud Shell 工作階段中，從 `slurm-gcp/tf/example` 資料夾執行下列指令：

```
terraform init
terraform apply -var-file=basic.tfvars
```

系統會根據您設定的配置，提示您接受所述動作。輸入「**yes**」即可開始部署。您也可以執行「`terraform plan`」，查看要部署的設定。

```
Do you want to perform these actions?
  Terraform will perform the actions described above.
  Only 'yes' will be accepted to approve.

Enter a value: yes
```

這項作業可能需要幾分鐘才能完成，請耐心等待。

注意：您可能需要授權 `gcloud` 發出 GCP API 呼叫。如果需要，請按一下「授權」。

部署完成後，您會看到類似以下的輸出內容：

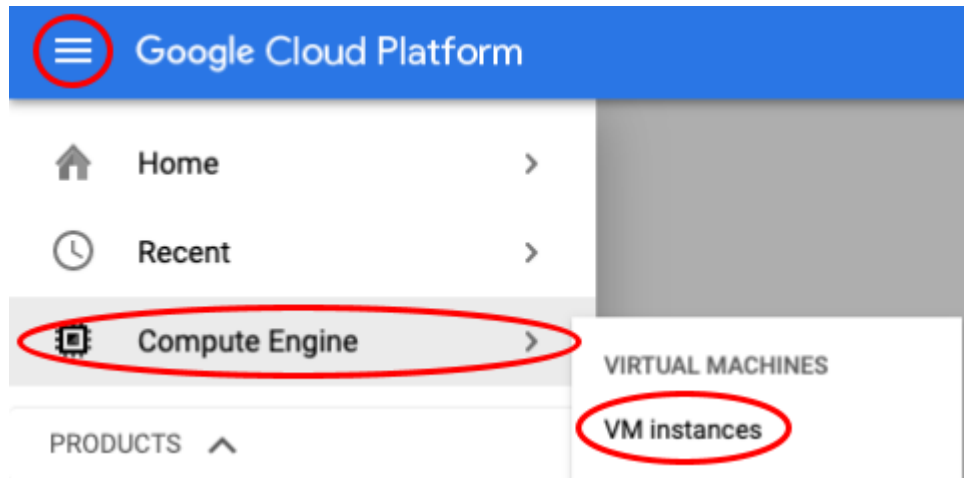
```
Apply complete! Resources: 8 added, 0 changed, 0 destroyed.

Outputs:

controller_network_ips = [
  [
    "10.0.0.2",
  ],
]
login_network_ips = [
  [
    "10.0.0.3",
  ],
]
```

確認 VM 執行個體建立作業

開啟導覽選單，依序選取「Compute Engine」>「VM 執行個體」。



您應該會看到列出的控制器和登入 VM 執行個體：

VM instances									
CREATE INSTANCE IMPORT VM REFRESH OPERATIONS SHOW INFO PANEL LEARN									
Filter Enter property name or value									
☐		Name ↑	Zone	Recommendations	In use by	Internal IP	External IP	Connect	
☐	✓	g1-controller	us-west1-b			10.0.0.2 (nic0)	None	SSH	⋮
☐	✓	g1-login0	us-west1-b			10.0.0.3 (nic0)	None	SSH	⋮

在「VM instances」(VM 執行個體) 下方，查看 Terraform 建立的兩個虛擬機器執行個體。

如果您修改了 **cluster_name** 欄位，名稱就會不同。

- **g1-controller**
- **g1-login0**

注意：如果您打算使用超過 100 個沒有外部 IP 的節點，建議為控制器執行個體使用大型機型 (n1-standard-8 以上)，以補償不足。

步驟 5 · 約 5 分鐘

5. 登入 Slurm 叢集

存取 Slurm 叢集

返回程式碼編輯器/Cloud Shell 分頁。執行下列指令登入執行個體，並將 `<ZONE>` 替換為 g1-login0 節點的可用區 (應為 `us-central1-b`)：

```
gcloud compute ssh g1-login0 --zone=<ZONE>
```

這個指令會將您登入 `g1-login0` 虛擬機器。

若要輕鬆存取登入節點，另一種方法是在「VM Instances」(VM 執行個體) 頁面中，按一下 g1-login0 VM 旁的「SSH」按鈕，開啟含有 SSH 連線的新分頁。

VM instances									
CREATE INSTANCE IMPORT VM REFRESH OPERATIONS SHOW INFO PANEL LEARN									
Filter Enter property name or value									
☐	☒	Name ↑	Zone	Recommendations	In use by	Internal IP	External IP	Connect	
☐	☒	g1-controller	us-west1-b			10.0.0.2 (nic0)	None	SSH	⋮
☐	☒	g1-login0	us-west1-b			10.0.0.3 (nic0)	None	SSH	⋮

如果您是第一次使用 Cloud Shell，系統可能會顯示如下訊息，要求您建立 SSH 金鑰：

```
WARNING: The public SSH key file for gcloud does not exist.
WARNING: The private SSH key file for gcloud does not exist.
WARNING: You do not have an SSH key for gcloud.
WARNING: SSH keygen will be executed to generate a key.
This tool needs to create the directory [/home/user/.ssh] before being
able to generate SSH keys.

Do you want to continue (Y/n)?
```

若要繼續，請輸入 **Y**。如果系統要求選取通關密語，請按兩次 **Enter** 鍵，將通關密語留空。

如果登入時出現以下訊息：

```
*** Slurm is currently being configured in the background. ***
A terminal broadcast will announce when installation and configuration is
complete.
```

請等待並在看到這則訊息前，不要繼續進行實驗室 (約 5 分鐘)：

```
*** Slurm login setup complete ***
```

看到上述訊息後，請登出 `g1-login0`，然後重新登入，繼續完成實驗室。若要這麼做，請按下 **CTRL + C** 鍵結束工作。

然後執行下列指令，登出執行個體：

```
exit
```

現在，請重新連線至登入 VM。執行下列指令登入執行個體，並將 `<ZONE>` 替換為 `g1-login0` 節點的可用區：

```
gcloud compute ssh g1-login0 --zone=<ZONE>
```

如上所述，您可能需要等待一到兩分鐘，才能連線並完成所有設定。

Slurm CLI 工具導覽

您現在已登入叢集的 Slurm 登入節點。這個節點專門用於使用者/管理員互動、排定 Slurm 作業，以及管理活動。

讓我們執行幾個指令，向您介紹 Slurm 指令列。

執行 **sinfo** 指令，查看叢集資源的狀態：

```
sinfo
```

下方顯示 `sinfo` 的範例輸出內容。`sinfo` 會回報叢集中可用的節點、這些節點的狀態，以及其他資訊，例如分割區、可用性和對這些節點施加的任何時間限制。

PARTITION	AVAIL	TIMELIMIT	NODES	STATE	NODELIST
debug*	up	infinite	10	idle~	g1-compute-0-[0-9]

您可以看到 10 個節點 (由偵錯分割區的「`max_node_count`」10 決定)，標示為「idle~」(節點處於閒置和未分配模式，準備啟動)。

接著，執行 **squeue** 指令，查看叢集佇列的狀態：

```
squeue
```

`squeue` 的預期輸出內容如下。`squeue` 會回報叢集的佇列狀態。包括叢集上排定的每項工作的工作 ID、工作指派的分區、工作名稱、啟動工作的使用者、工作狀態、工作執行時間 (以實際時間計算)，以及工作分配到的節點。我們沒有任何正在執行的工作，因此這項指令的內容是空白。

JOBID	PARTITION	NAME	USER	ST	TIME	NODES	NODELIST(REASON)
-------	-----------	------	------	----	------	-------	------------------

Slurm 的「`srun`」和「`sbatch`」指令可用於執行已加入佇列的工作。「`srun`」會執行平行工作，可用於做為 `mpirun` 的包裝函式。「`sbatch`」用於將批次工作提交至 Slurm，且可呼叫 `srun` 一次或多次，並採用不同設定。「`sbatch`」可以採用批次指令碼，也可以搭配 `-wrap` 選項使用，從指令列執行整個工作。

讓我們執行工作，看看 Slurm 的運作方式，並在佇列中取得工作！

6. 執行 Slurm 工作並擴充叢集

執行 Slurm 工作並擴充叢集

Slurm 叢集現在已開始運作，接下來請執行工作並擴大叢集。

「sbatch」指令用於執行 Slurm 批次指令和指令碼。我們來執行簡單的 sbatch 指令碼，在自動調整規模的 VM 上執行「hostname」。

登入 g1-login0 後，執行下列指令：

```
sbatch -N2 --wrap="srun hostname"
```

這個指令會執行 Slurm 批次指令。這會指定 sbatch 使用「-N」選項執行 2 個節點。此外，這項指令也指定每個節點都會在「--wrap」選項中執行「srun hostname」指令。

根據預設，sbatch 會將輸出內容寫入指令執行所在工作目錄的「slurm-%j.out」，其中 %j 會根據 [Slurm 檔案名稱模式](#) 替換為工作 ID。在本範例中，sbatch 是從使用者的 /home 資料夾執行，而該資料夾是預設由控制器代管的 NFS 型共用檔案系統。這樣一來，運算節點就能視需要共用輸入和輸出資料。在正式環境中，工作儲存空間應與 /home 儲存空間分開，以免影響叢集作業的效能。您可以在 tfvars 檔案的「network_storage」選項中指定個別儲存空間掛接點。

使用 sbatch 指令列執行 sbatch 指令碼後，系統會傳回已排定的工作的「工作 ID」，例如：

```
Submitted batch job 2
```

我們可以透過 sbatch 指令傳回的作業 ID，追蹤及管理作業執行和資源。**執行**下列指令，查看 Slurm 工作佇列：

```
squeue
```

您可能會看到執行的工作列於下方：

JOBID	PARTITION	NAME	USER	ST	TIME	NODES	NODELIST(REASON)
2	debug	g1-compute-0-[0-1]	username	R	0:10	2	g1-compute-0-[0-1]

由於我們未佈建任何運算節點，Slurm 會根據工作需求自動建立運算執行個體。這項程序會自動進行，有兩大優點。首先，在 HPC 叢集中，您通常需要手動佈建節點、設定軟體、將節點整合至叢集，然後部署工作，但使用 Cloud HPC Toolkit 即可省去這些步驟。其次，這項功能會縮減閒置未使用的節點，直到節點數量達到下限為止，因此可協助使用者節省費用。

您可以執行 **sinfo** 指令，查看啟動中的 Slurm 叢集：

```
sinfo
```

這會顯示 squeue 中列出的節點處於「alloc#」狀態，表示節點正在分配中：

```

PARTITION AVAIL  TIMELIMIT  NODES  STATE NODELIST
debug*      up    infinite      8  idle~ g1-compute-0-[2-9]
debug*      up    infinite      2  alloc# g1-compute-0-[0-1]

```

您也可以在 Google Cloud 控制台的「VM 執行個體」部分，查看新佈建的節點。系統會花費幾分鐘的時間啟動節點並執行 Slurm，然後將工作分配給新分配的節點。VM 執行個體清單很快就會如下所示：

☐	☒	Name ↑	Zone	Recommendations	In use by	Internal IP	External IP	Connect
☐	☒	g1-compute-0-0	us-west1-b			10.1.0.2 (nic0)	None	SSH ▾ ⋮
☐	☒	g1-compute-0-1	us-west1-b			10.1.0.3 (nic0)	None	SSH ▾ ⋮
☐	☒	g1-controller	us-central1-f			10.0.0.2 (nic0)	None	SSH ▾ ⋮
☐	☒	g1-login0	us-central1-f			10.0.0.3 (nic0)	None	SSH ▾ ⋮

節點開始執行工作後，執行個體就會進入「alloc」狀態，表示工作已分配給工作：

```

PARTITION AVAIL  TIMELIMIT  NODES  STATE NODELIST
debug*      up    infinite      8  idle~ g1-compute-0-[2-9]
debug*      up    infinite      2  alloc g1-compute-0-[0-1]

```

作業完成後，系統就不會在 `squeue` 中列出該作業，且 `sinfo` 中的「alloc」節點會返回「閒置」狀態。每隔一兩分鐘執行「`squeue`」，直到工作完成為止。

輸出檔案 `slurm-%j.out` 會寫入 NFS 共用的 `/home` 資料夾，並包含主機名稱。開啟或 `cat` 輸出檔案 (通常是 `slurm-2.out`)，輸出檔案內容會包含：

```

g1-compute-0-0
g1-compute-0-1

```

你已執行工作並擴充 Slurm 叢集，做得好！

注意：`sbatch` 是執行 Slurm 批次指令碼的 Slurm 指令。[瞭解詳情](#)。

`srun` 是 Slurm 指令，可並行執行指令。[瞭解詳情](#)。

`squeue` 是 Slurm 佇列監控指令列工具。這個頁面會列出所有正在執行的工作，以及與這些工作相關聯的資源。[瞭解詳情](#)。

`sinfo` 是 Slurm 指令，會列出 Slurm 叢集的相關資訊。這會列出叢集中的 Slurm 分割區、可用性、時間限制和節點目前狀態。[瞭解詳情](#)。

步驟 7 · 約 0 分鐘

7. 執行 MPI 工作

現在，讓我們在節點之間執行 [MPI](#) 工作。登入 g1-login0 後，使用 wget 下載以 C 程式設計語言編寫的 MPI 程式：

```
wget https://raw.githubusercontent.com/mpitutorial/mpitutorial/gh-  
pages/tutorials/mpi-hello-world/code/mpi_hello_world.c
```

如要使用 OpenMPI 工具，請執行下列指令載入 OpenMPI 模組：

```
module load openmpi
```

我們會使用「mpicc」工具編譯 MPI C 程式碼。執行下列指令：

```
mpicc mpi_hello_world.c -o mpi_hello_world
```

這會將 C 程式碼編譯為機器碼，以便透過 Slurm 在叢集中執行程式碼。

接下來，請使用偏好的文字編輯器建立名為「**helloworld_batch**」的 sbatch 指令碼：

```
vi helloworld_batch
```

輸入 **i** 進入 vi 插入模式。

複製並貼上下列文字到檔案中，建立簡單的 sbatch 指令碼：

```
#!/bin/bash  
#  
#SBATCH --job-name=hello_world  
#SBATCH --output=hello_world-%j.out  
#  
#SBATCH --nodes=2  
  
srun mpi_hello_world
```

按下 Esc 鍵並輸入「:wq」(不含引號)，即可儲存並結束程式碼編輯器。

這個指令碼會定義 Slurm 批次執行環境和工作。首先，執行環境定義為 Bash。接著，指令碼會先使用「#SBATCH」行定義 Slurm 選項。工作名稱定義為「hello_world」。

輸出檔案會設為「hello_world_%j.out」，其中 %j 會根據 [Slurm 檔案名稱模式](#) 替換為工作 ID。這個輸出檔案會寫入 sbatch 指令碼執行的目錄。在本範例中，這是使用者的 /home 資料夾，也就是以 NFS 為基礎的共用檔案系統。這樣一來，運算節點就能視需要共用輸入和輸出資料。在正式環境中，工作儲存空間應與 /home 儲存空間分開，以免影響叢集作業的效能。

最後，這個指令碼應執行的節點數量定義為 2。

定義選項後，系統會提供可執行的指令。這個指令碼會使用 srun 指令以平行方式執行 mpi_hello_world 程式碼，而 srun 指令是 mpirun 指令的替代指令。

然後使用 `sbatch` 指令列執行 `sbatch` 指令碼：

```
sbatch helloworld_batch
```

執行 `sbatch` 會傳回已排定的工作的 **工作 ID**，例如：

```
Submitted batch job 3
```

這會在 2 個節點上執行 `hostname` 指令 (每個節點各一項工作)，並將輸出內容列印到 **hello_world-3.out** 檔案。

由於我們已佈建 2 個節點，這項工作會快速執行。

監控 **squeue**，直到工作完成且不再列出為止：

```
squeue
```

完成後，開啟或 `cat hello_world-3.out` 檔案，確認該檔案在 `g1-compute-0-[0-1]` 上執行：

```
Hello world from processor g1-compute-0-0, rank 0 out of 2 processors
Hello world from processor g1-compute-0-1, rank 1 out of 2 processors
```

閒置 5 分鐘後 (可透過 YAML 的 `suspend_time` 欄位或 `slurm.conf` 的 `SuspendTime` 欄位設定)，系統會取消配置動態佈建的運算節點，以釋出資源。您可以定期執行 `sinfo`，觀察叢集大小是否回歸為 0，藉此驗證這點：

PARTITION	AVAIL	TIMELIMIT	NODES	STATE	NODELIST
debug*	up	infinite	10	idle~	g1-compute-0-[0-9]

請嘗試啟動更多執行個體 (最多可達叢集部署區域的配額上限)，並執行不同的 MPI 應用程式。

8. 結論

恭喜！您已在 Google Cloud Platform 上建立 Slurm 叢集，並使用最新功能自動調度叢集資源，滿足工作負載需求。您可以使用這個模型執行各種工作，只要在 Slurm 中要求節點，就能在幾分鐘內擴充至數百個執行個體。

如要繼續瞭解如何在 GCP 上使用 Slurm，請務必繼續完成「[透過 Slurm 建構聯合 HPC 叢集](#)」程式碼研究室。本程式碼實驗室將引導您在雲端設定兩個聯盟 Slurm 叢集，代表您可能實現的多叢集聯盟，無論是在地端部署或雲端。

您是否正在使用 Slurm 的全新 GCP 原生功能建構酷炫項目？有任何問題嗎？想提出功能建議嗎？歡迎立即透過 [Google Cloud 高效能運算解決方案網站](#) 與 Google Cloud 團隊聯絡，或在 [Google Cloud 和 Slurm 討論群組](#) 中與我們即時通訊！

清除 Terraform 部署作業

登出 Slurm 節點：

```
exit
```

請先讓自動調度資源的節點縮減，再刪除部署作業。您也可以手動刪除這些節點，方法是針對每個執行個體執行「`gcloud compute instances delete <執行個體名稱>`」，或是使用主控台 GUI 選取多個節點，然後按一下「刪除」。

完成後，您可以登出 `g1-login0`，然後從 Google Cloud Shell 執行下列指令，輕鬆清理 Terraform 部署作業：

```
cd ~/slurm-gcp/tf/examples/basic
terraform destroy -var-file=basic.tfvars
```

系統提示時，請輸入 **yes** 繼續操作。這項作業可能需要幾分鐘，請耐心等待。

刪除專案

如要清理，只要刪除專案即可。

- 在導覽選單中選取「IAM 與管理」
- 然後按一下子選單中的「設定」
- 按一下「刪除專案」文字旁的垃圾桶圖示
- 按照提示操作

涵蓋內容

- 瞭解如何使用 Terraform 在 GCP 上部署 Slurm。
- 如何在 GCP 上使用 Slurm 執行工作。
- 如何查詢叢集資訊，以及監控 Slurm 中執行的工作。
- 瞭解如何使用 Slurm on GCP 自動調整節點大小，以配合特定工作參數和需求。
- 瞭解如何在 GCP 上的 Slurm 編譯及執行 MPI 應用程式。

尋找 Slurm 支援

如需在測試或正式環境中使用這些整合功能的支援服務，請透過 SchedMD 的聯絡頁面 (<https://www.schedmd.com/contact.php>) 直接與他們聯絡。

您也可以使用下列疑難排解指南：

- GCP 上的 Slurm 疑難排解指南：<https://github.com/SchedMD/slurm-gcp#troubleshooting>
- SchedMD 的疑難排解指南：<https://slurm.schedmd.com/troubleshoot.html>

最後，您也可以將問題發布至 Google Cloud 和 Slurm 討論群組，網址為：<https://groups.google.com/g/google-cloud-slurm-discuss>

注意：這個社群討論群組並非 Slurm 官方支援管道，我們無法保證會回覆。我們將盡力提供支援，但不會透過傳統的 Slurm 支援管道提供。如需官方支援，請與 [SchedMD](#) 聯絡。

瞭解詳情

Google Cloud Platform 連結

- [Google Cloud HPC 聯絡頁面](#)
- [Google Cloud Compute Engine](#)
- [Google Cloud Deployment Manager](#)
- [Google Cloud 和 Slurm 討論群組](#)

Slurm 連結

- [SchedMD 首頁](#)
- [SchedMD 聯絡頁面](#)
- [Slurm 快速入門指南](#)
- [Slurm MAN 頁面](#)
- [Slurm 指令摘要 \(PDF\)](#)
- [Slurm 會計指南](#)
- [Slurm 疑難排解指南](#)
- [在 GCP 上的 Slurm 安裝應用程式](#)
- [Slurm 使用者討論群組](#)

意見回饋

請[按這裡](#)提交本程式碼研究室的意見回饋。填寫意見回饋表單只需不到 5 分鐘。感謝您！
